SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

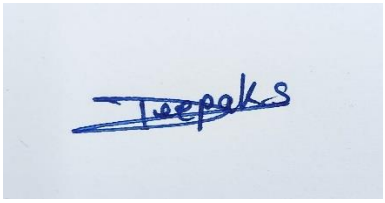
9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I S Deepak_1921260077 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Q1. Boundary Failures in Online Fund Transfers

10 Marks

1. Scenario Analysis

An online banking system fails during fund transfers, mostly at the **minimum and maximum transfer limits**. This pattern is the classic signature of a **boundary defect**: a relational operator written as `<` instead of `<=` (or the reverse), or a limit stored with the wrong precision. Defects that cluster at limits show that the existing test cases never probed the edges of the input domain.

Assumptions (stated for a concrete design):

- Allowed amount per transaction: minimum **₹100.00**, maximum **₹1,00,000.00**, up to 2 decimal places.
- A transfer is allowed only if the source account is active and the available balance is at least the amount.
- Amount must be numeric; blank, negative, alphabetic and special-character inputs must be rejected with an error message.

2. Evaluation of the Current Test Case Design

Observation	Weakness in current strategy	Effect
Failures appear at min/max limits	No boundary-focused test cases exist	Off-by-one defects reach production
Mostly random amounts used	Random values rarely land on the exact limits	Limit logic is untested
Valid inputs dominate	Few invalid-class and negative tests	Weak error-handling coverage
Amount and balance tested separately	Interaction (amount vs balance) is missed	Overdraft / wrongly-blocked transfers
No traceability to requirements	Coverage cannot be measured	False confidence in test adequacy

Table 1.1 Evaluation of the current design

3. Equivalence Class Partitioning (ECP)

Class	Input condition	Range / description	Type
EC1	Amount below minimum	0.00 ≤ amount < 100.00	Invalid
EC2	Amount within limits	100.00 ≤ amount ≤ 1,00,000.00	Valid
EC3	Amount above maximum	amount > 1,00,000.00	Invalid
EC4	Non-numeric / negative / blank	'abc', '-500', '' , '@#'	Invalid
EC5	Balance sufficient	balance ≥ amount	Valid
EC6	Balance insufficient	balance < amount	Invalid
EC7	Too many decimals	100.505 (3 decimal places)	Invalid

Table 1.2 Equivalence classes for fund transfer

4. Boundary Value Analysis (BVA)

Using the **three-value BVA** (just below, on, just above each boundary), the test cases are redesigned as follows (balance = ₹2,00,000 and account active, unless stated):

TC	Amount (₹)	Expected result	Technique
T1	99.99	Rejected - below minimum	BVA (min - 1 paisa)
T2	100.00	Accepted	BVA (min)
T3	100.01	Accepted	BVA (min + 1 paisa)
T4	50,000.00	Accepted	ECP - nominal value of EC2
T5	99,999.99	Accepted	BVA (max - 1 paisa)
T6	1,00,000.00	Accepted	BVA (max)
T7	1,00,000.01	Rejected - above maximum	BVA (max + 1 paisa)
T8	-500 / abc / blank	Rejected - invalid format	ECP (EC4)
T9	100.505	Rejected - precision error	ECP (EC7)
T10	5,000 with balance 5,000	Accepted, balance becomes 0	BVA on balance (equal)
T11	5,000 with balance 4,999.99	Rejected - insufficient funds	BVA on balance (below)

Table 1.3 Redesigned test cases

5. Effectiveness of Random Testing

If amounts are drawn uniformly from ₹0.00 to ₹2,00,000.00 there are 2,00,00,001 possible values (in paise). Only six of them are boundary values, so the chance that one random test hits a boundary is about 6 / 2,00,00,001 = **0.000003 %**. Even 1,000 random tests would almost never touch the limits where the failures occur.

Criterion	Random testing	Structured (ECP + BVA)
Targets known fault zones	No - by chance only	Yes - boundaries chosen deliberately
Repeatability	Low (different values each run)	High (fixed, documented values)
Coverage measurable	No	Yes - every class and boundary traced
Finds unexpected defects	Sometimes	Limited to modelled faults
Cost for strong confidence	Very high	Low - about 11 to 15 cases

Table 1.4 Random vs structured testing

Decision on random testing

Random testing should be **replaced as the primary technique** and **retained only as a small supplementary activity** (for example, automated random or fuzz inputs on the transfer API) to discover defects that no model predicted. It must never be the basis for claiming adequacy.

6. Proposed Structured and Optimized Approach

1. **Analyse requirements** and fix the limits, precision and balance rules in writing.
2. **Apply ECP** to remove redundant tests: one representative per class.
3. **Apply BVA** to amount and balance limits, giving the 11 cases above.
4. **Add a decision table** for combinations: amount valid / balance sufficient / account active (8 rules, 1 positive and 7 negative).
5. **Automate** the suite and run it in regression after every change to transfer logic.

6. **Retain a short exploratory / random session** for residual risk and monitor production defects to feed new cases back.

7. Justification and Conclusion

Defects live at boundaries, so BVA places tests exactly where programmers most often err, while ECP guarantees every behaviour class is exercised without duplication. The combined suite gives **maximum defect detection with a minimum number of test cases**, is repeatable, and offers measurable adequacy (100 % of classes and boundaries covered). Replacing random testing with this structured approach should remove the boundary failures and raise confidence in every release.

Q2. Coverage Gaps in a Healthcare Module (White Box) — 10 Marks

1. Evaluation of the Testing Approach Used

The module passed testing yet critical defects appeared after deployment. This indicates that testing was **requirement-driven and happy-path oriented**: tests confirmed that normal prescriptions work, but nobody measured *which parts of the code* were executed. In healthcare the unexecuted parts are exactly the dangerous ones (paediatric limits, allergy checks, error handling).

The module is modelled conceptually as a **prescription-safety check** with the following logic:

```
1  read record, drug, dose
2  if (!record.valid)            -> 3 return ERROR
4  if (patient.age < 12)
5       limit = pediatricLimit(weight)
6  else limit = adultLimit(drug)
7  if (dose > limit)             -> 8 return REJECT
9  for each allergy:             (loop)
10      if (drug matches allergy) -> 11 return BLOCK
12 return APPROVE
13 exit
```

2. Conceptual Control Flow Graph

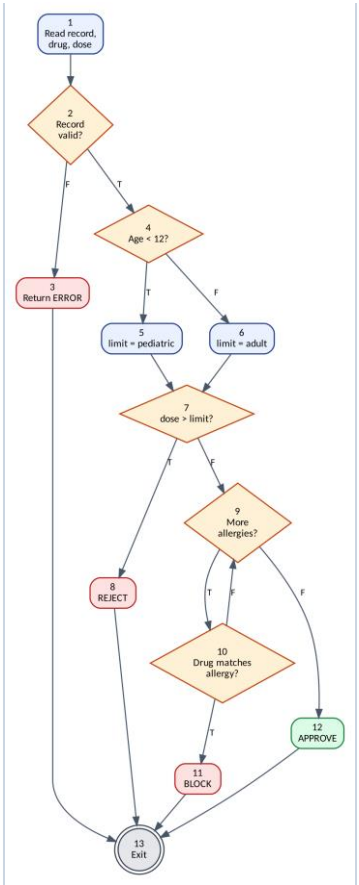


Figure 2.1 Control Flow Graph of the prescription-safety module

Graph metrics: Nodes N = 13, Edges E = 17, Predicate nodes = 5 (nodes 2, 4, 7, 9, 10). Cyclomatic complexity **V(G) = E - N + 2 = 17 - 13 + 2 = 6** (check: predicates + 1 = 5 + 1 = 6). Therefore **six independent paths** must be tested.

3. Independent Paths and Missing Paths

Path	Node sequence	Scenario	Status in original tests
P1	1-2-3-13	Invalid record	Covered
P2	1-2-4-5-7-8-13	Child, overdose	MISSING
P3	1-2-4-6-7-8-13	Adult, overdose	Covered
P4	1-2-4-5-7-9-12-13	Child, safe, no allergies	MISSING
P5	1-2-4-6-7-9-10-11-13	Adult, allergy match	MISSING
P6	1-2-4-5-7-9-10-9-12-13	Allergy list checked, no match	MISSING

Table 2.1 Independent paths (the adult safe path 1-2-4-6-7-9-12-13 was also exercised)

4. Critical Assessment of Coverage Achieved

Assume the team ran three tests: (a) invalid record, (b) adult with a safe dose and no allergies, (c) adult overdose. Coverage is then:

Criterion	Covered	Total	Percentage	Missed elements
Statement (nodes)	10	13	77 %	Nodes 5, 10, 11 - paediatric limit and the allergy BLOCK
Branch (edges)	11	17	65 %	Edges 4→5, 5→7, 9→10, 10→11, 10→9, 11→13
Path (basis paths)	2 of 6 (plus 1 adult safe path)	6	about 50 %	P2, P4, P5, P6

Table 2.2 Coverage assessment

> **Key gap**
> Even 77 % statement coverage hid the two riskiest features: **allergy blocking** and **paediatric dose limits**. Statement coverage alone is a weak adequacy criterion because it ignores untaken branches and loop behaviour.

5. Improved Testing Strategy

TC	Input	Path	Expected result
W1	Invalid record	P1	ERROR
W2	Age 8, 20 kg, dose above paediatric limit	P2	REJECT
W3	Age 40, dose above adult limit	P3	REJECT
W4	Age 8, safe dose, allergy list empty (0 iterations)	P4	APPROVE
W5	Age 40, safe dose, drug = penicillin, allergy list has penicillin	P5	BLOCK
W6	Age 8, safe dose, 1 allergy that does not match	P6	APPROVE
W7	Age 40, safe dose, 3 allergies, last one matches	Loop: many iterations	BLOCK
W8	Age 11 / 12 and dose = limit / limit + 1	Boundary of D2, D3	Correct limit chosen; limit + 1 rejected

Table 2.3 Improved white-box test suite

- Adopt **100 % branch coverage** and all six basis paths as the minimum exit criterion.

- Add **loop tests** (0, 1, 2, many iterations) for the allergy loop.
- Combine with black-box BVA on age and dose so structure and requirements both guide the tests.
- Measure coverage automatically in the build pipeline and block release below the target.

6. Impact on Reliability and Risk

Executing every decision outcome and independent path means each safety rule is run at least once with both truth values, so hidden faults in dosage, age and allergy logic are exposed **before** patients are affected. Measurable coverage also gives an auditable adequacy argument, reducing clinical, legal and reputational risk and improving the module's reliability after every release.

Q3. Input Combination Errors in an E-Commerce System

10 Marks

1. Scenario Analysis

The retail system accepts **price, quantity and discount**. Testers validated each field and a few typical combinations, but defects remained in *specific combinations* of inputs. Black-box testing that treats fields independently cannot reveal errors caused by **interaction between inputs**.

Assumed specification:

- Price: ₹1.00 to ₹50,000.00. Quantity: integer 1 to 100. Discount: 0 % to 50 %.
- Bulk rule: if quantity ≥ 10, an extra 5 % discount is added.
- Effective discount = min(discount + bulk, 50 %). Total = price × quantity × (1 - effective discount).

2. Limitations of the Current Black-Box Approach

Limitation	Explanation
Single-field focus	Each field is checked alone, so the bulk-rule interaction (quantity vs discount) is never exercised.
Typical combinations only	A few 'normal' combinations leave most of the input space untested.
No boundaries	Values such as quantity 9/10 and discount 45/46 (where the cap starts to act) are skipped.
No systematic combination model	Without a decision table the team cannot prove that all rules were covered.
Invalid combinations ignored	Multiple simultaneously invalid fields (and which error is shown) are not tested.

Table 3.1 Limitations

3. Equivalence Class Partitioning

Field	Valid class	Invalid classes
Price	V1: 1 ≤ p ≤ 50,000	I1: p < 1 (0, negative) I2: p > 50,000 I3: non-numeric / blank
Quantity	V2: 1 ≤ q ≤ 100 (integer)	I4: q < 1 I5: q > 100 I6: decimal / non-numeric
Discount	V3: 0 ≤ d ≤ 50	I7: d < 0 I8: d > 50 I9: non-numeric

Table 3.2 Equivalence classes

4. Boundary Value Analysis

Field	Boundary test values (three-value BVA)
Price (₹)	0.99, 1.00, 1.01, 49,999.99, 50,000.00, 50,000.01
Quantity	0, 1, 2, 9, 10, 11, 99, 100, 101
Discount (%)	-1, 0, 1, 45, 46, 49, 50, 51

Table 3.3 BVA values (other fields held at nominal values)

Quantity **9 / 10** and discount **45 / 46** are *derived boundaries*: they come from the bulk rule and the 50 % cap, not from the input ranges, and are typically missed.

5. Decision Table Testing

Table A - validity of inputs (Y = valid):

Condition / action	R1	R2	R3	R4	R5	R6	R7	R8
Price valid	Y	Y	Y	Y	N	N	N	N
Quantity valid	Y	Y	N	N	Y	Y	N	N
Discount valid	Y	N	Y	N	Y	N	Y	N
Action	Calculate total	Discount error	Qty error	Qty + disc. error	Price error	Price + disc. error	Price + qty error	All errors

Table 3.4 Validity decision table (8 rules)

Table B - calculation interaction (price ₹1,000):

Rule	Quantity ≥ 10	Discount > 45	Test (q, d)	Effective discount	Expected total (₹)
C1	N	N	9, 40	40 %	5,400
C2	Y	N	10, 40	45 %	5,500
C3	Y	Y	10, 46	min(51, 50) = 50 %	5,000
C4	N	Y	9, 50	50 %	4,500

Table 3.5 Interaction decision table

6. Revised Test Design Strategy

1. **Single-field BVA** with other fields nominal: 6 + 9 + 8 = 23 tests (reduced to about 18 after removing duplicates).
2. **Validity decision table:** 7 additional negative rules (R2-R8).
3. **Interaction decision table:** 4 rules including derived boundaries.
4. **Pairwise combination of ECP classes:** the three fields have 4, 4 and 3 classes (48 full combinations); pairwise testing needs only about **16 tests** while covering every pair of classes.
5. **Regression automation** with data-driven tests; exploratory session for residual risk.

Total ≈ 45 designed tests, compared with a full BVA grid of 6 × 9 × 8 = 432 combinations, yet every class, boundary, rule and pair is covered.

7. Justification

- **Improved defect detection:** combinational faults (bulk + cap) are targeted directly by decision-table rules.
- **Completeness:** the decision tables enumerate every condition combination, so no rule is forgotten; the pairwise set guarantees all two-way interactions.
- **Efficiency:** ≈ 45 tests instead of hundreds, with full traceability to requirements.

Conclusion

Combining ECP, BVA and decision-table testing converts an unstructured, field-by-field approach into a **complete and economical combination strategy**, directly addressing the missed input-combination errors.

Q4. Complete Path Testing for Nested Conditions and Loops

10 Marks

1. Evaluation of the Current White-Box Approach

The team reports only **partial path coverage** for a module with nested conditions and a loop. Typical symptoms are that tests exercise the *main flow* repeatedly (valid items, one loop pass) while inner `else` branches, zero-iteration and multi-iteration behaviour are skipped. The approach lacks (i) a formally drawn CFG, (ii) a computed count of independent paths, and (iii) a coverage target.

A representative module (order total calculation) is analysed:

```
1  total = 0; i = 0
2  while (i < n)
3      if (item[i].valid)
4          if (item[i].qty > 0)
5              total += price * qty
6          else flag++
7      else log_invalid()
8      i++
9  if (total > LIMIT)
10     total += surcharge
11 return total
```

2. Control Flow Graph and Gaps in Its Construction

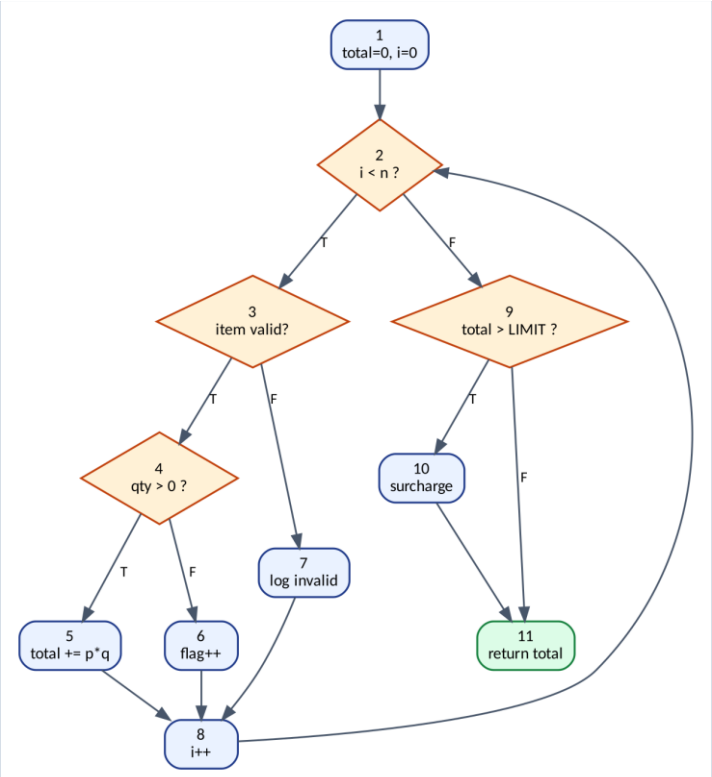


Figure 4.1 CFG with nested decisions and a loop

Common construction gaps: (a) the loop back-edge 8→2 omitted; (b) `else` edges 3→7 and 4→6 not drawn, so those paths are never planned; (c) the zero-iteration exit 2→9 forgotten; (d) the compound nested `if` merged into one decision node.

Metrics: N = 11, E = 14, predicates = 4 (nodes 2, 3, 4, 9). **V(G) = 14 - 11 + 2 = 5** (= 4 + 1). Five independent paths are required.

3. Impact of Missing Independent Paths

Independent path	Node sequence	Meaning
P1	1-2-9-11	Empty list, loop skipped (n = 0)
P2	1-2-3-4-5-8-2-9-11	Valid item, normal total
P3	1-2-3-4-6-8-2-9-11	Valid item with quantity ≤ 0
P4	1-2-3-7-8-2-9-11	Invalid item skipped
P5	1-2-3-4-5-8-2-9-10-11	Total above LIMIT, surcharge

Table 4.1 Basis path set

Suppose existing tests exercised only P1, P2 and P5. Edge coverage is 10 of 14 = **71 %**; the four unexecuted edges (3→7, 7→8, 4→6, 6→8) hide the invalid-item and zero-quantity logic. A defect there, for example `flag` never reset or an invalid item corrupting `total`, would go undetected. Because loops multiply the number of paths, missing one independent path can silently leave an entire family of behaviours untested.

> **Feasibility note**
> Path 1-2-9-10-11 (zero iterations then surcharge) is **infeasible** because total = 0 cannot exceed a positive LIMIT. Infeasible paths must be identified and documented so that they are not counted as missed coverage.

4. Proposed Complete Path Testing Strategy

TC	Input data	Path	Expected outcome
T1	n = 0	P1	Total = 0, no surcharge
T2	n = 1, valid, qty 2, price 100, LIMIT 1000	P2	Total = 200
T3	n = 1, valid, qty 0	P3	flag = 1, total = 0
T4	n = 1, invalid item	P4	Item logged and skipped
T5	n = 1, valid, qty 20, price 100 (total 2000 > LIMIT)	P5	Surcharge added
T6	n = 3 mixed: valid, invalid, qty 0	P2 + P4 + P3 in one run	Correct total, flag and log
T7	n = max (e.g. 100) valid items	Loop at maximum	No overflow, correct total
T8	n = max + 1	Loop boundary	Rejected / handled

Table 4.2 Path and loop-boundary test suite

For the loop, apply the standard loop strategy: **0, 1, 2, typical, max - 1, max, max + 1** iterations. If loops are nested, test the innermost loop first with outer loops at minimum values, then work outward.

5. Justification Using Coverage Criteria and Expected Outcomes

- **Statement coverage 100 %** (every node 1-11 executed by T1-T5).
- **Branch coverage 100 %** (all 14 edges traversed; T3 and T4 supply the missing edges).
- **Basis path coverage 5/5** = V(G), so every independent path, and by linear combination every other feasible path, is tested.

- **Loop coverage** with zero, one, many and boundary iterations.

The expected outcome is that residual defects in nested decisions and loop handling are detected before release, and the team can *demonstrate* adequacy with metrics instead of estimates.

Q5. Hybrid Black-Box and White-Box Strategy for a Financial Application

10 Marks

1. Critical Analysis of the Current Integration

The project uses both black-box and white-box testing, yet defects reach production. This shows that *using* both techniques is not the same as *integrating* them. Typical weaknesses:

Weakness	Explanation	Consequence
Techniques run in silos	Black-box tests were never checked against code coverage	Untested branches remain unnoticed
Weak adequacy criterion	Statement coverage accepted as 'done'	Untaken decision outcomes and loops missed
BVA only on inputs	Derived values (interest, EMI, fees) not bounded	Calculation boundaries fail in production
No feedback loop	Coverage gaps do not generate new test cases	Gaps persist release after release
Coverage metrics misused	Percentages reported without risk weighting	Critical modules treated like trivial ones

Table 5.1 Why defects escape

2. Evaluation of Test Adequacy Criteria and Metrics

Test adequacy criteria were **not properly applied**: black-box adequacy (every equivalence class and boundary covered) and white-box adequacy (branch, path, condition coverage) were neither defined nor combined. A suite can satisfy one and still fail the other, for example by covering all loan-amount classes yet never executing the 'EMI exceeds 40 % of income' branch.

3. Proposed Hybrid Testing Strategy

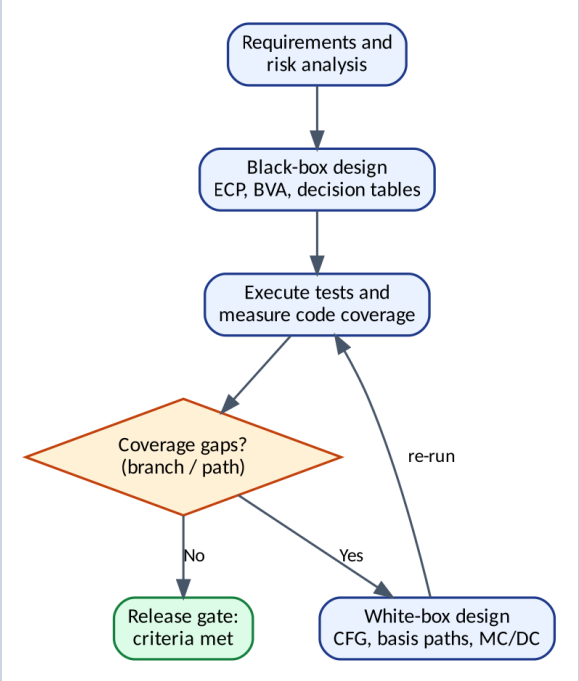


Figure 5.1 Hybrid test design and feedback loop

Phase	Activity	Output / gate
1. Requirements	Identify risk-ranked functions (loan approval, interest, transfers)	Risk register
2. Black box	ECP and BVA on inputs; decision table on rule combinations	Specification-based test set
3. Measure	Run tests with a coverage tool	Statement / branch / path report
4. White box	Build CFG; derive basis paths; add MC/DC for compound decisions	Tests for every uncovered path
5. Gate	Exit criteria check	Release only if all criteria met

Table 5.2 Phases of the hybrid strategy

4. Worked Example: Loan Approval Module

Black-box part (assumed ranges: loan ₹50,000-₹50,00,000, credit score 300-900, approve if score ≥ 650 and EMI ≤ 40 % of income):

TC	Input	Expected	Technique
B1	Loan 49,999	Reject - invalid amount	BVA
B2	Loan 50,000, score 650, EMI 40 %	Approve	BVA (all on boundary)
B3	Score 649	Reject - credit	BVA
B4	Score 750, EMI 41 %	Manual review	ECP / decision table

Table 5.3 Black-box tests

White-box part: the CFG of the approval logic is drawn and the independent paths identified.

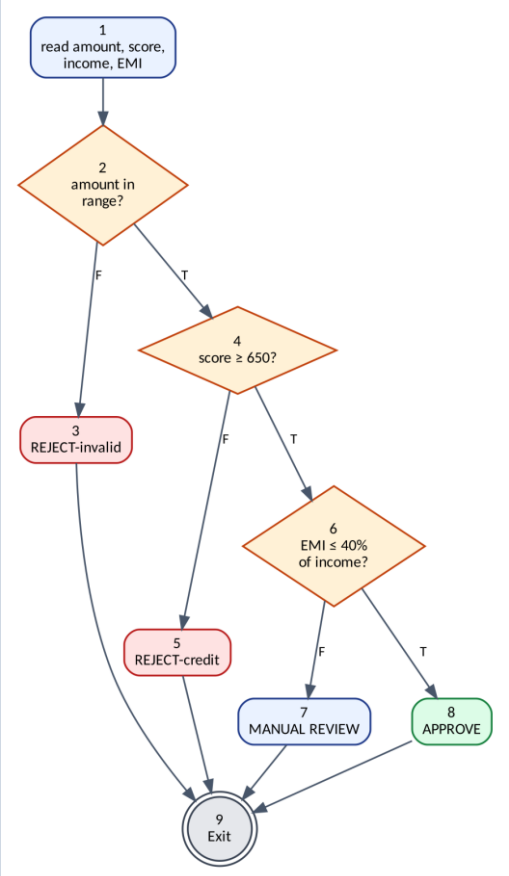


Figure 5.2 CFG of the loan approval logic

N = 9, E = 11, **V(G) = 11 - 9 + 2 = 4**. Basis paths: P1 = 1-2-3-9, P2 = 1-2-4-5-9, P3 = 1-2-4-6-7-9, P4 = 1-2-4-6-8-9. Tests B1-B4 map exactly to P1-P4; had one been missing, the coverage report in phase 3 would expose it and phase 4 would add a test. For the compound condition (score ≥ 650 AND EMI ≤ 40 %) **MC/DC** needs n + 1 = 3 tests, showing that each condition independently affects the decision.

5. Justification: Better Coverage and Defect Prevention

- **Complementary strengths:** black-box finds missing or wrong functionality; white-box finds unexecuted logic. Together they cover specification and implementation.
- **Measured adequacy:** exit criteria such as 100 % of classes and boundaries, ≥ 95 % branch coverage, all basis paths of critical modules, and MC/DC for approval decisions.
- **Feedback loop:** every coverage gap becomes a new test, so quality improves each cycle.
- **Defect prevention:** CFG and decision-table reviews expose logic and requirement errors early, and automated regression keeps fixed defects from returning.

Conclusion

A hybrid strategy with **defined criteria, coverage measurement and a feedback loop** converts two parallel activities into one coherent process and is the most reliable way to stop production defects in a financial application.

Q6. Bonus Calculation Failures at 5, 10 and 15 Years of Service

10 Marks

1. Critical Analysis of the Failure

Employees with **exactly 5, 10 and 15 years** of service received wrong bonuses. All three values are the *lower limits* of service bands, so the failure pattern points to a **boundary condition error**, most probably a relational operator mistake such as `years > 5` instead of `years >= 5`. Employees at the boundary are then paid at the lower band's rate.

Assumed specification: monthly bonus = salary × band rate × rating factor.

Years of service	Band rate	Rating	Factor
0 - 4	2 %	1	0.0
5 - 9	4 %	2	0.5
10 - 14	6 %	3	1.0
15 - 40	8 %	4	1.25
< 0 or > 40: invalid	-	5	1.5

Table 6.1 Assumed bonus rules (rating valid range 1 to 5)

Example (salary ₹50,000, rating 3): at 5 years the correct bonus is ₹2,000 (4 %), but the faulty code pays ₹1,000 (2 %).

2. Evaluation of the Current Test Design

- Tests used mid-band values (for example 3, 7, 12, 20 years) which pass even with the faulty operator.
- No test targeted the exact band limits or the values just beside them.
- Rating and service were not combined at their limits.
- Invalid service values (negative, above 40) and invalid ratings (0, 6) were not checked.

3. Redesign Using Equivalence Class Partitioning

Parameter	Class	Range	Validity
Years	Y1	years < 0	Invalid
	Y2	0 - 4	Valid
	Y3	5 - 9	Valid
	Y4	10 - 14	Valid
	Y5	15 - 40	Valid
	Y6	years > 40	Invalid
Rating	R0 / R6	rating < 1 or > 5	Invalid
	R1-R5	each valid rating is its own class (distinct factor)	Valid

Table 6.2 Equivalence classes

4. Redesign Using Boundary Value Analysis

Salary ₹50,000 and rating 3 (factor 1.0), unless stated:

TC	Years / rating	Expected bonus (₹)	Tests
1	-1 / 3	Error message	Lower invalid boundary
2	0 / 3	1,000	Lower valid boundary
3	4 / 3	1,000	Band 1 upper limit
4	**5** / 3	**2,000**	Boundary (detects > defect)
5	6 / 3	2,000	Boundary + 1
6	9 / 3	2,000	Band 2 upper limit
7	**10** / 3	**3,000**	Boundary (detects > defect)
8	14 / 3	3,000	Band 3 upper limit
9	**15** / 3	**4,000**	Boundary (detects > defect)
10	40 / 3	4,000	Upper valid boundary
11	41 / 3	Error message	Upper invalid boundary
12	7 / 0 and 7 / 6	Error message	Rating invalid boundaries
13	7 / 1, 7 / 5	0 and 3,000	Rating valid boundaries
14	5 / 5, 10 / 5, 15 / 5	3,000, 4,500, 6,000	Combined boundary interaction

Table 6.3 BVA-based test suite (14 tests)

The three bold rows would **fail immediately** on the faulty code (actual 1,000 / 2,000 / 3,000), so the defect would be found in the first test run.

5. Why ECP + BVA are Superior to Exploratory Testing Here

Aspect	Exploratory testing	ECP + BVA
Fault targeting	Depends on tester intuition; boundaries may be skipped	Boundaries are systematically derived
Repeatability / audit	Poor - difficult to prove what was tested	Documented, repeatable evidence for audit
Precision of oracle	Tester estimates 'looks right'	Exact expected values computed from rules
Regression	Hard to automate	Easy to automate
Financial risk	Rounding / off-by-one errors easily missed	Directly aimed at off-by-one errors

Table 6.4 Comparison

Conclusion

Payroll logic is **deterministic, rule-based and financially sensitive**. ECP removes redundancy while BVA targets the exact limits where programming errors occur. A compact set of 14 tests would have caught the 5, 10 and 15 year defect before release; exploratory testing can complement it but cannot replace it for this logic.

Q7. CFG, Branch Gaps and MC/DC for a Temperature Controller

10 Marks

1. Logic Under Test

The controller reads a temperature and two sensor-status flags. An alarm must sound when the temperature is critical **or** when **both sensors have failed**; otherwise heater or cooler is controlled. Assumed thresholds: LOW = 20 °C, HIGH = 60 °C, CRIT = 80 °C.

```
1  read T, s1Ok, s2Ok
2  if ( (T > CRIT)  ||  (!s1Ok && !s2Ok) )     // decision with conditions A, B, C
5      ALARM(); shutdown();
6  else if (T < LOW)   7 heater ON;
8  else if (T > HIGH)  9 cooler ON;
10 else idle;
11 log(); exit;
```

2. Control Flow Graph (condition-level)

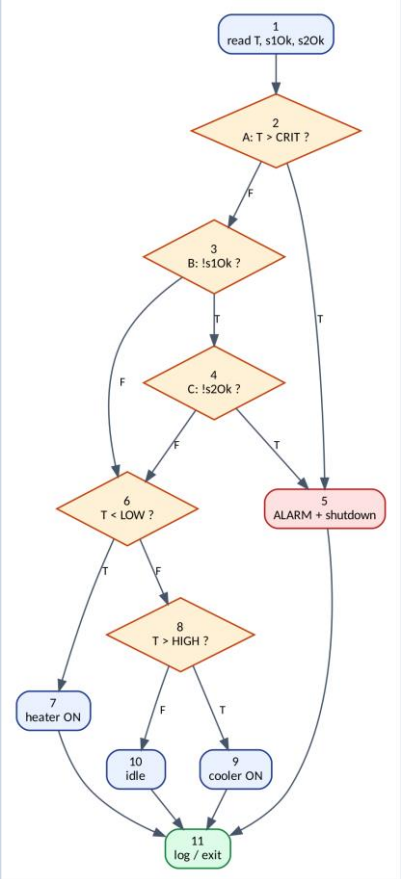


Figure 7.1 CFG with the compound decision split into conditions A, B and C

Each condition of the compound decision has its own node because short-circuit evaluation creates a separate branch for every condition. N = 11, E = 15, predicate nodes = 5 (nodes 2, 3, 4, 6, 8). **V(G) = 15 - 11 + 2 = 6** (= 5 + 1), so six basis paths are required:

Path	Nodes	Scenario
P1	1-2-5-11	T above CRIT, alarm

Path	Nodes	Scenario
P2	1-2-3-4-5-11	Both sensors failed, alarm (the failing sequence)
P3	1-2-3-4-6-8-10-11	Only sensor 2 failed, idle
P4	1-2-3-6-7-11	Sensors OK, T below LOW, heater
P5	1-2-3-6-8-9-11	Sensors OK, T above HIGH, cooler
P6	1-2-3-6-8-10-11	Sensors OK, normal temperature, idle

Table 7.1 Basis paths

3. Evaluation of the Gaps in Branch Coverage

The post-release failure occurred in the malfunction sequence in which **both sensors fail while the temperature is normal**. The earlier tests probably covered 'T > CRIT' (alarm) and normal operation only. That already gives **100 % decision coverage** of the alarm decision (true and false observed), yet:

- Edges **3→4, 4→5 and 4→6** were never traversed, so conditions B and C were never evaluated as true.
- Path P2, the only path where the alarm depends on sensor failure, was never run.
- A fault such as writing `&&` instead of `||` between A and the sensor term, or `!s10k || !s20k`, could not be detected.

> **Lesson**
> Decision coverage only requires the whole expression to be true once and false once. It **does not prove that each condition matters**. That is why safety-critical standards (DO-178C, IEC 61508 and similar) demand MC/DC.

4. White-Box Strategy Achieving 100 % MC/DC

For decision **A || (B && C)** with three conditions, MC/DC needs at least n + 1 = **4 tests**. Each condition must have an *independence pair*: two tests that differ only in that condition and produce different decision outcomes.

Test	A (T>CRIT)	B (!s1Ok)	C (!s2Ok)	Decision	Concrete input	Independence pair for
M1	0	0	1	F	T=50, s1 OK, s2 failed	A (with M2), B (with M3)
M2	1	0	1	T	T=85, s1 OK, s2 failed	A
M3	0	1	1	T	T=50, both failed	B, C
M4	0	1	0	F	T=50, s1 failed, s2 OK	C

Table 7.2 MC/DC test set (with masking MC/DC for short-circuit code)

Pairs: **A** = (M1, M2); **B** = (M1, M3); **C** = (M3, M4). Two more tests complete branch coverage of the control logic: **M5** T = 10, sensors OK → heater (P4) and **M6** T = 70, sensors OK → cooler (P5). The six tests M1-M6 traverse **all 15 edges**, so 100 % statement, branch and MC/DC coverage are achieved with only six tests. (M1 also exercises normal idle behaviour, P6.)

5. Impact on Safety-Critical Reliability

- The exact failure (both sensors lost) is now a **mandatory, traceable test** (M3).
- Every condition is proven to affect the alarm decision independently, which removes masked faults.
- Six tests instead of the $2^3 = 8$ exhaustive combinations x threshold tests keeps cost low as conditions grow (n + 1 versus 2^n).

- Supports certification evidence: coverage results map directly to safety standards.

Q8. Promo Code Combinations and Negative Totals

10 Marks

1. Limitations of Testing Inputs in Isolation

Each promo code was probably tested alone and produced a correct total. A **negative Total Amount** appears only when several discounts are applied together and their sum exceeds the fare. Isolated testing cannot reveal it because the defect is a property of the *combination*, not of any single input.

- Interactions (percentage + flat + tier benefit) are never exercised.
- The order of applying discounts is not verified.
- The lower limit of the total amount (floor / cap rule) is untested.
- The number of combinations grows quickly, so ad-hoc selection leaves most untested.

Assumed rules: SAVE20 = 20 % off base fare; FLAT500 = ₹500 flat off; Gold tier = extra 5 % off base fare; discounts are added; **total combined discount is capped at 80 % of the base fare** (so total never falls below 20 % of fare); the same code cannot be applied twice; expired or invalid codes are rejected.

2. Cause-Effect Analysis

ID	Causes (inputs)	ID	Effects (outputs)
C1	Percentage code SAVE20 applied	E1	Add percentage discount
C2	Flat code FLAT500 applied	E2	Add flat discount
C3	User tier is Gold	E3	Add tier discount
C4	Combined discount exceeds 80 % cap	E4	Cap discount at 80 %
C5	Duplicate or invalid code	E5	Reject code with error message

Table 8.1 Causes and effects

3. Decision Table (all valid and invalid combinations)

Conditions: **P1** percentage code, **P2** flat code, **G** Gold tier, **X** combined discount above cap. $2^4 = 16$ combinations; the 4 with X = Y and no flat code are **infeasible** (20 % + 5 % can never reach 80 %), leaving 12 valid rules, plus 2 invalid-input rules.

Rule	P1	P2	G	X	Fare ₹	Expected total ₹	Buggy total ₹
R1	N	N	N	N	5,000	5,000	5,000
R2	N	N	Y	N	5,000	4,750	4,750
R3	Y	N	N	N	5,000	4,000	4,000
R4	Y	N	Y	N	5,000	3,750	3,750
R5	N	Y	N	N	5,000	4,500	4,500
R6	N	Y	Y	N	5,000	4,250	4,250
R7	Y	Y	N	N	5,000	3,500	3,500
R8	Y	Y	Y	N	5,000	3,250	3,250
R9	N	Y	N	Y	500	100	0
R10	Y	Y	N	Y	500	100	-100

Rule	P1	P2	G	X	Fare ₹	Expected total ₹	Buggy total ₹
R11	Y	Y	Y	Y	500	100	-125
R12	N	Y	Y	Y	500	100	-25
R13	duplicate code applied twice				5,000	Error; 2nd code rejected	-
R14	expired / invalid code				5,000	Error; no discount	-

Table 8.2 Decision table with computed expected results

R9-R12 show how the defect occurs: on a ₹500 fare, FLAT500 alone reduces the total to 0 and each added discount pushes it negative. With the cap, the total is 20 % of the fare = ₹100.

4. Boundary Tests Around the Cap

TC	Fare (₹)	Codes	Discount computed	Expected total (₹)
B1	625	FLAT500	500 = exactly 80 % of 625	125 (on the boundary)
B2	624	FLAT500	500 > 499.20 (80 %)	124.80 (capped)
B3	626	FLAT500	500 < 500.80	126

Table 8.3 Boundary tests at the 80 % cap

5. Optimized Test Set

Rather than testing every fare x code x tier x order combination, the optimized set contains:

- **12 feasible decision-table rules** R1-R12 (R1-R8 below the cap, R9-R12 at the cap).
- **2 invalid-input rules** R13 and R14.
- **3 boundary tests** B1-B3 on the cap.
- Total **17 tests**, each unique in the behaviour it verifies; the 4 infeasible combinations are removed, and equivalent fares (₹1,000 or ₹4,000 behave like ₹5,000) are not repeated.

6. Justification

The decision table guarantees **systematic coverage of every combination** while removing infeasible or redundant ones, which maximizes defect detection per test case. The cap boundaries pinpoint the exact threshold at which negative totals begin. The table also doubles as documentation: the business can review each rule, and any new promo type adds columns rather than ad-hoc tests.

Conclusion

Testing promo codes in isolation hides combination defects. **Decision Table / Cause-Effect testing** reveals them, and the optimized 17-test set covers all valid and invalid combinations without redundancy, preventing negative totals.

Q9. Path Testing of a Nested-Loop Migration Tool

10 Marks

1. Scenario Analysis

The migration tool validates records inside two nested loops: the outer loop walks the tables and the inner loop walks the records of each table. The team reports **high statement coverage** but production runs hang in **infinite loops**. This means the tests executed every line, yet never explored how the loops *terminate*.

Assumed logic (a typical legacy design): a record that fails validation is put back in the queue for another attempt, and the queue length `records[i]` is increased. No retry limit exists.

```
1  i = 0
2  while (i < tables) {
3      j = 0
4      while (j < records[i]) {
5          if (valid(r[j])) {   migrate(r[j]);              }
6          else             {   requeue(r[j]); records[i]++; }
7          j++
8      }
9      i++
10 }
```

2. Evaluation of the Current White-Box Approach

- **Statement coverage is the weakest criterion.** One table with two records (one invalid but repairable) executes every statement, so coverage reads 100%.
- It never checks **how many times** a loop runs, so zero-iteration, boundary-iteration and non-terminating cases stay hidden.
- The invalid branch only needs to run once; a record that stays invalid forever (so `records[i]` grows as fast as `j`) is never tried, and this is exactly the production crash.
- No independent-path analysis was done, so combinations of outer-loop and inner-loop behaviour were never planned.

3. Control Flow Graph and Cyclomatic Complexity

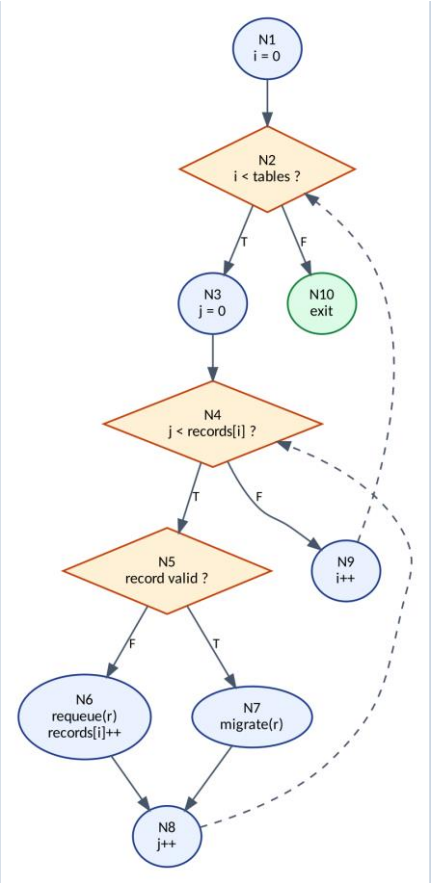


Figure 1: CFG of the nested validation loops (dashed edges are loop-back edges)

Metric	Calculation	Result
Edges / Nodes	E = 12, N = 10	-
V(G) = E - N + 2	12 - 10 + 2	**4**
V(G) = predicates + 1	N2, N4, N5 are decisions: 3 + 1	**4**
V(G) = regions	Bounded regions 3 + outer region 1	**4**

4. Proposed Path-Testing Strategy: Independent Paths

Path	Node sequence	Test data	Expected result
P1	N1-N2-N10	tables = 0	Nothing migrated, clean exit
P2	N1-N2-N3-N4-N9-N2-N10	1 table, 0 records	Inner loop skipped, table closed
P3	N1-N2-N3-N4-N5-N7-N8-N4-N9-N2-N10	1 table, 1 valid record	Record migrated
P4	N1-N2-N3-N4-N5-N6-N8-N4-N9-N2-N10	1 table, 1 invalid record	Requeued and retried, then stops by retry cap

5. Loop Boundary Conditions

Loop	Test	Value	Purpose
Inner	Skip / once / twice	0, 1, 2 records	Entry, exit and re-entry

Loop	Test	Value	Purpose
Inner	Typical	50 records	Normal operation
Inner	Max-1, Max, Max+1	9,999 / 10,000 / 10,001 (assumed limit)	Limit handling, overflow of counter
Inner	**Never-valid record**	Corrupt record + retry cap = 3	Must stop with error, not hang (timeout guard)
Outer	Skip / once / many	0, 1, 20 tables	Outer iteration control
Nested	Innermost first, outer at minimum; then outer, inner typical; then both at max	1x0, 1x50, 20x50, max x max	Beizer nested-loop method

Key insight

Statement coverage proves that code was *run*, not that loops *finish*. The defect lives in the loop bound, so only loop-boundary and path tests can expose it.

6. Justification Using Cyclomatic Complexity

- V(G) = 4 gives the **minimum number of tests** (4) needed to execute every independent path once, so the base suite is small and complete.
- A value of 4 is well below the common limit of 10, so the module is simple enough to test exhaustively; if a retry counter adds decisions, V(G) rises and the suite must grow with it.
- Path P4 plus the never-valid test targets the exact infinite-loop cause, while P1 and P2 cover zero-iteration defects.

7. Conclusion

Replacing statement coverage by **basis-path testing plus loop-boundary tests** (backed by a timeout guard) gives measurable coverage of every decision outcome and of loop termination, which removes the infinite-loop crashes.

Q10. State Transition Testing of the Real-Time Order Tracker

10 Marks

1. Scenario Analysis

The tracker moves through **Ordered, Preparing, Out for Delivery and Delivered**. Users report that the app *gets stuck* when the driver cancels mid-delivery. The four-state model has **no state and no event** for this situation, so the app has nowhere to go; the requirement itself is incomplete.

2. Evaluation of the Current Black-Box Strategy

- **ECP and BVA test inputs, not history.** They check single values (order amount, address length) but cannot test *which event happens in which state*.
- Tests probably followed only the happy path Ordered -> Preparing -> Out for Delivery -> Delivered.
- Events such as driver cancel, restaurant delay or timeout were never listed, and illegal events were never tried.

3. Redesigned State Model

Two states are added: **Reassigning Driver** (waiting for a new driver) and **Cancelled** (terminal, with refund).

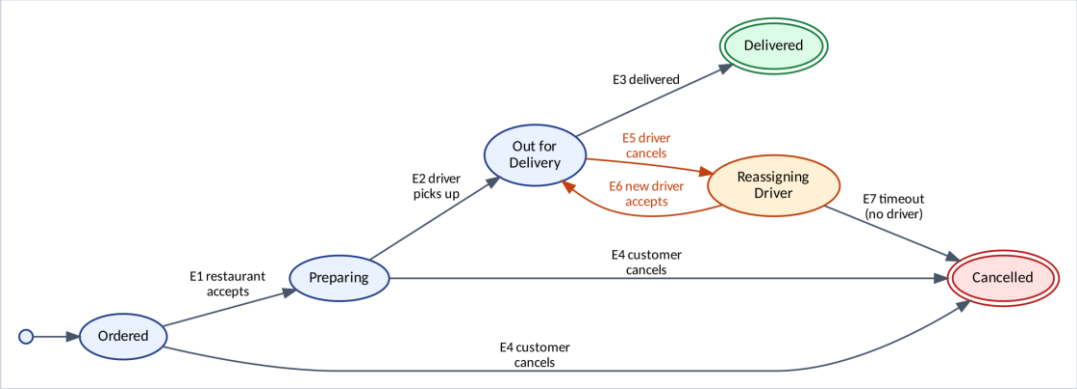


Figure 1: State-transition diagram with the driver-cancel flow (orange)

4. State Transition Table (all legal and illegal transitions)

Each cell shows the next state; X marks an **illegal** event that must be rejected without changing state.

State	E1 Accept	E2 Pickup	E3 Deliver	E4 Cust. cancel	E5 Driver cancel	E6 New driver	E7 Timeout
Ordered	Preparing	X	X	Cancelled	X	X	X
Preparing	X	Out for Delivery	X	Cancelled	X	X	X
Out for Delivery	X	X	Delivered	X (too late)	Reassigning	X	X
Reassigning	X	X	X	Cancelled	X	Out for Delivery	Cancelled
Delivered	X	X	X	X	X	X	X
Cancelled	X	X	X	X	X	X	X

5. Redesigned Test Scenarios

ID	Scenario (start state -> event sequence)	Expected result
ST1	Ordered -> E1 -> E2 -> E3	Delivered; customer notified (happy path)
ST2	Out for Delivery -> E5	Reassigning; app shows "Finding a new driver", not frozen
ST3	Reassigning -> E6	Back to Out for Delivery with new driver and new ETA
ST4	Reassigning -> E7 (after 10 min)	Cancelled; full refund; customer message
ST5	Out for Delivery -> E5 -> E6 -> E5 -> E6 -> E3	Repeated cancel loop works; Delivered
ST6	Ordered -> E4 / Preparing -> E4	Cancelled; refund (partial if food prepared)
ST7	Out for Delivery -> E4 (illegal)	Rejected with message; state unchanged
ST8	Delivered -> E5, Cancelled -> E1 (illegal)	Rejected; terminal states stay terminal
ST9	Ordered -> E3 (skip states)	Rejected; no jump to Delivered
ST10	E5 and E3 arrive together (race)	One event wins; consistent final state

6. Why State Transition Testing Covers What ECP and BVA Miss

- The same input (a "cancel" event) gives **different results in different states**; ECP/BVA treat it as one value.
- It tests **sequences** (0-switch and 1-switch), so loops such as cancel, reassign, cancel again are found.
- It forces **illegal transitions** to be tested, which is where "stuck" behaviour hides.
- Coverage is measurable: all states, all valid transitions (9 here), all 33 illegal cells (6 states x 7 events = 42 cells, 9 of them legal).

> **Key insight**
> The bug was a *missing state*, not a wrong value. Modelling the workflow as a state machine exposes missing states and events before coding even starts.

7. Conclusion

State Transition Testing turns the dynamic workflow into a checkable model, ensures every legal and illegal transition is exercised and removes the stuck-tracker defect.

Q11. Hybrid Testing Strategy for Multi-Factor Authentication

10 Marks

1. Scenario Analysis

The authentication module combines **password strength, OTP timing and IP location**. Brute-force vulnerabilities still reached production, so the tests proved that *valid* logins work but not that *attacks* fail.

Assumed logic (used for the white-box part):

```
1 if (attempts[u] >= 5)            return LOCKED
2 if (!meetsPolicy(pwd))          return REJECT_WEAK
3 if (!pwdMatches(u,pwd))  { attempts[u]++; return DENIED }
4 if (now - otp.issued > 30s)    return OTP_EXPIRED
5 if (otp.value != input)        return DENIED      // attempts not incremented
6 if (!trustedRegion(ip))        return STEP_UP
7 attempts[u] = 0;               return GRANTED
```

2. Assessment of the Black-Box and White-Box Methods Used

Method	What it achieved	Gap
Black box (ECP/BVA)	Checked password length, OTP at 29/30/31 s, trusted and untrusted IPs	Tested *inputs one by one*; never repeated attempts, volume or rate
White box (statement/branch)	Every line of the login function executed	Line 5 returns without counting a failure, yet is "covered"; coverage cannot show a *missing* statement
Integration	Each factor tested separately	Interaction (OTP failure not feeding the lockout) never checked

3. Error Guessing Component (attacker mindset)

- **OTP brute force:** 000000 to 999999 within the 30 s window with no lockout (the exact flaw at line 5).
- **Credential stuffing** and password spraying over many usernames from one IP.
- **Lockout bypass:** change case of the username, add spaces, open parallel sessions.
- **OTP replay** after success; reuse of an expired OTP.
- **IP spoofing** using the `X-Forwarded-For` header; switching to a trusted-region VPN.
- **Race condition:** 100 simultaneous requests before the counter updates.
- **Injection and long inputs** in username and password fields.

4. Formal Path Analysis

Six decisions give V(G) = 6 + 1 = **7 independent paths**; each is executed with a security expectation.

Path	Decision outcomes	Test data	Expected result
P1	D1 = T	5 earlier failures	LOCKED (even with correct password)
P2	D1 = F, D2 = T	Password "abc"	REJECT_WEAK
P3	D2 = F, D3 = T	Wrong password	DENIED; counter + 1

Path	Decision outcomes	Test data	Expected result
P4	D3 = F, D4 = T	OTP age 31 s	OTP_EXPIRED
P5	D4 = F, D5 = T	Wrong OTP	DENIED; **counter must increase** (fails today)
P6	D5 = F, D6 = T	Correct credentials, foreign IP	STEP_UP
P7	all F	All correct, trusted IP	GRANTED; counter reset

5. The Proposed Hybrid Strategy

1. **Threat model** the module (STRIDE) to list attack goals.
2. **Error guessing** turns each attack into a concrete test (table below).
3. **Path analysis** guarantees all 7 paths, and the *attack* tests are mapped to the path they stress.
4. **Automation:** scripts send 1,000 attempts and assert lockout after attempt 5.
5. **Review of coverage:** branch coverage plus a security checklist (OWASP ASVS) before release.

ID	Attack test	Path	Expected result
E1	10 wrong OTPs in a row	P5	Account locked at 5th failure
E2	Same OTP used twice	P7	Second use rejected
E3	50 usernames x same password	P3	IP rate-limited / CAPTCHA
E4	`ADMIN` vs `admin` lockout	P1	Same account, same counter
E5	100 parallel logins	P3/P5	Counter exact (no race)
E6	Forged `X-Forwarded-For`	P6	Real client IP is used

6. Justification

- Path analysis gives **systematic completeness**, while error guessing supplies **creative attack coverage**; each covers the other's blind spot.
- The hybrid finds the real defect (line 5 not counting failures), which neither statement coverage nor ECP could see.
- Automated brute-force tests make the lockout rule **measurable and repeatable** in every release, improving the security posture.

Conclusion

Formal paths prove every decision is tested; error guessing proves the decisions cannot be abused. Together they close the brute-force gap.

Q12. ECP for Tiered Subscriptions and Expiry-Day Upgrades

10 Marks

1. Scenario Analysis

The e-learning platform offers **Free, Basic and Premium** tiers whose access depends on the subscription date and the course type. A defect appears only when a user upgrades **exactly on the expiration day**, the one date where "active" and "expired" meet.

Assumptions: a subscription is valid *through the end of its expiry date D*; a 7-day grace period follows; Free never expires; access rules are Free -> free courses, Basic -> free + basic, Premium -> all courses.

2. Evaluation of the Current Testing Criteria

- Tests probably used "typical" dates (middle of the term) and ignored **boundary dates** (D-1, D, D+1).
- Date logic was treated as one input; tier, date and course type were not **combined**.
- The "<" versus "<=" error in `today <= expiry` can only be seen on D itself.

3. Equivalence Class Partitioning

Input	Valid classes	Invalid classes
Subscription tier	V1 Free, V2 Basic, V3 Premium	I1 unknown tier, I2 null / empty
Date vs expiry D	V4 before D (active, D-1 or earlier), V5 **on D (expiry day)**, V6 D+1 to D+7 (grace), V7 beyond D+7 (lapsed)	I3 date before start date, I4 impossible date (e.g. 30-02)
Course type	V8 free, V9 basic, V10 premium	I5 course not found
Upgrade path	V11 Free->Basic, V12 Free->Premium, V13 Basic->Premium	I6 downgrade by upgrade call, I7 same tier

4. Structured Test Design

Assumed data: Basic subscription, start 2026-01-01, expiry **D = 2026-12-31**.

ID	Class	Action date	Action	Expected result
T1	V4	2026-12-30	Upgrade Basic -> Premium	Success; premium access; new expiry +365 d
T2	V5	**2026-12-31**	Upgrade Basic -> Premium	Success; **no gap in access, no double charge, no downgrade to Free**
T3	V6	2027-01-01	Upgrade (grace day 1)	Success; treated as renewal + upgrade
T4	V6	2027-01-07	Upgrade (last grace day)	Success
T5	V7	2027-01-08	Upgrade	Account Free; upgrade starts a new subscription
T6	V5	2026-12-31	Open a Basic course	Access allowed (expiry day is still valid)
T7	V5 / V10	2026-12-31	Open a Premium course as Basic	Denied; offer upgrade
T8	V7 / V9	2027-01-08	Open a Basic course	Denied (lapsed -> Free)
T9	V11	2026-06-15	Free -> Basic	Success; new expiry set correctly

ID	Class	Action date	Action	Expected result
T10	V12	2026-06-15	Free -> Premium	Success; all courses open
T11	I3	2025-12-31	Any action	Rejected: before start date
T12	I4	2026-02-30	Any action	Rejected: invalid date
T13	I1	2026-06-15	Tier = "Gold"	Rejected with error
T14	I6	2026-06-15	Premium -> Basic via upgrade	Rejected: use downgrade flow
T15	V5, leap	2028-02-29	Upgrade on a leap-day expiry	Success; next expiry date computed correctly

Key insight

T2 is the bug-finding case: one representative from the **expiry-day class** is enough, but only if that class is defined separately from "active".

5. Justification of Completeness

- Every valid and invalid class of tier, date, course type and upgrade path is hit at least once (T1-T15), so **class coverage = 100%**.
- Boundary dates D-1, D, D+1, D+7 and D+8 verify both sides of every boundary.
- Tier x date-class pairs (3 x 4 = 12 combinations) are covered by T1-T10, which guarantees **seamless transitions** at each change of status.
- A pass on T2 shows that the user keeps access continuously, the main business expectation.

6. Conclusion

Treating **expiry day** as its own equivalence class, and combining it with tier and course type, finds the upgrade-on-expiry error and gives users a smooth, gap-free upgrade experience.

Q13. Branch and Condition Coverage for the Smart-Home Dimmer

10 Marks

1. Scenario Analysis

The lighting controller adjusts brightness from an ambient-light sensor (`lux`) using **switch-case statements** and a **complex Boolean expression**. Hardware faults and wrong dimming levels would show up only after integration with real lamps, so logic errors must be removed earlier.

```
1  switch (mode) {
2    case AUTO:
3      if (lux < 200 && (occupied || override))
4           level = 100 - lux/2;
5      else level = 0;
6      break;
7    case MANUAL:
8      level = userLevel;          // missing break
9    case NIGHT:
10     level = 10;  break;
11   default:
12     level = 0;  raiseError();
13 }
```

2. Risks of Relying Only on Statement Coverage

- Four tests (AUTO true, AUTO false, MANUAL, invalid mode) reach every statement, so the report says 100%.
- It does **not** force both outcomes of each decision; for example the NIGHT case alone is never selected, and the `else` path may stay unchecked if the test expects the wrong value.
- **Hidden fall-through:** MANUAL runs line 9 as well and overwrites `userLevel` with 10. Statement coverage marks the lines as covered; only a test with an *expected value* check (70 expected, 10 obtained) exposes it.
- Individual conditions `occupied` and `override` could be wrong (for example `&&` coded instead of `||`) without changing statement coverage.
- Risk for hardware: a wrong level or an unhandled sensor range is sent straight to the lamp driver.

3. Branch Coverage: Missing Logical Paths

There are 4 switch outcomes + 2 `if` outcomes = **6 branches**.

Test	mode	lux	occupied / override	userLevel	Branch	Expected level
B1	AUTO	100	T / F	-	if = True	50
B2	AUTO	500	T / F	-	if = False	0
B3	MANUAL	-	-	70	case MANUAL	70 (finds fall-through: gets 10)
B4	NIGHT	-	-	-	case NIGHT	10
B5	9 (invalid)	-	-	-	default	0 + error raised

4. Condition Coverage and MC/DC for A && (B || C)

A = `lux < 200`, B = `occupied`, C = `override`. Four tests give **MC/DC**, in which each condition independently changes the decision.

Test	A	B	C	Decision	Shows independence of
C1	T	T	F	T	B (with C3)
C2	T	F	T	T	C (with C3), A (with C4)
C3	T	F	F	F	B (with C1), C (with C2)
C4	F	F	T	F	A (with C2)

These four tests also give **condition coverage** (A, B, C each true and false) and decision coverage (True and False), using n + 1 = 4 tests instead of 2^3 = 8.

5. Improved White-Box Strategy

1. Cover all **6 branches** (B1-B5 above) with expected-value checks.
2. Add the **MC/DC set C1-C4**.
3. Add **boundary tests** for the sensor: lux = 199, 200, 201 and also invalid readings (-1, 65535) to confirm `default` / range handling.
4. Fix the missing `break` and re-run to show the repair.
5. Run the same tests on a **hardware-in-the-loop** rig with a simulated lux sensor.

6. Reduction of Integration Risk

- Each decision outcome and each condition is proven to be tested, so incorrect levels are caught **before** they reach the lamp driver.
- Fall-through and invalid-mode handling are verified, preventing commands the hardware cannot interpret.
- Boundary sensor tests expose noise and range errors that cause flicker.
- MC/DC gives high confidence with only 4 tests for the Boolean expression, so the suite stays fast enough for regression.

Conclusion

Moving from statement coverage to branch, condition and MC/DC coverage detects the fall-through and the Boolean-logic errors, lowering hardware-software integration failures.

Q14. Robustness BVA for Shipping-Cost Input Validation

10 Marks

1. Scenario Analysis

The logistics system computes the shipping cost from **weight, destination zone and delivery speed**. Testers never tried **zero or negative weights**, so the system accepted them and produced zero or negative shipping cost. A cost of 0 or below means direct revenue loss and corrupt billing data.

Assumed specification: weight 0.1 kg to 30.0 kg; zones 1 to 5; speed = Standard, Express or Overnight.

2. Analysis of the Failure

- Testing was confined to **valid values only** ("normal" BVA), so values outside the range were never sent.
- The weight rule `weight >= 0.1` was either missing in the code or applied after the cost formula.
- Since `cost = rate x weight`, a negative weight gives a **negative cost** and zero gives a **free shipment**.
- Test data came from sample orders, which never contain invalid values.

3. Equivalence Classes

Input	Valid class	Invalid classes
Weight	0.1 <= w <= 30.0	w < 0 (negative), w = 0, 0 < w < 0.1, w > 30.0, non-numeric
Zone	1, 2, 3, 4, 5	zone < 1 (0, -1), zone > 5 (6), blank
Speed	Standard, Express, Overnight	unknown ("Drone"), blank

4. Robustness BVA: Weight Test Cases

Robustness testing adds the points just **outside** each boundary (min-1 and max+1) to the usual five values.

ID	Weight (kg)	Boundary position	Class	Expected result
W1	-5.0	Far negative	Invalid	Error "weight must be positive"; no cost
W2	-0.1	Just negative	Invalid	Error; no cost
W3	0	Zero	Invalid	Error; no free shipment
W4	0.09	Min - 1 step	Invalid	Error "below minimum weight"
W5	0.10	Min	Valid	Cost = minimum charge
W6	0.11	Min + 1 step	Valid	Cost calculated
W7	15.0	Nominal	Valid	Cost calculated
W8	29.9	Max - 1 step	Valid	Cost calculated
W9	30.0	Max	Valid	Cost calculated
W10	30.1	Max + 1 step	Invalid	Error / route to freight service

5. Zone and Speed Boundaries (other inputs held at valid values)

ID	Input	Value	Expected result
Z1	Zone	0	Rejected
Z2	Zone	1	Accepted (min)
Z3	Zone	5	Accepted (max)
Z4	Zone	6	Rejected
S1	Speed	Standard / Express / Overnight	Accepted with correct multiplier
S2	Speed	"Drone" / blank	Rejected
M1	Combination	weight = 0 and zone = 6	Both errors reported; no cost

```
if (weight < 0.1 || weight > 30.0)  return ERROR_WEIGHT;   // validate first
if (zone < 1 || zone > 5)          return ERROR_ZONE;
cost = rate[zone][speed] * weight;           // cost is always > 0
```

6. Why Negative Scenarios Are Necessary

- **Financial integrity:** invalid values must never reach the cost formula; otherwise the company loses money or issues negative invoices.
- **Robustness:** real users, APIs and scanners send wrong data; the system must reject it gracefully, not crash or guess.
- **Security:** unchecked inputs are the entry point for overflow and injection attacks.
- **Defect detection:** boundary values are where programmers make off-by-one mistakes (`>` instead of `>=`); testing min-1 and max+1 detects them.
- **Data quality:** rejected input keeps wrong weights out of reports and warehouse planning.

Conclusion

A system is only as robust as its response to bad input. Robustness BVA with min-1, max+1, zero and negative values closes the validation gap and prevents loss-making shipments.

Q15. Basis Path Testing for the Parallel Imaging Module

10 Marks

1. Scenario Analysis

The imaging software splits a dataset of `N` voxels into **blocks of 64** processed by parallel workers, and a leftover part (`N mod 64`) is processed separately. Results are wrong **only when N is prime**. A prime N larger than 64 always leaves a non-zero remainder, and (except for N = 2) an **odd** one, so the code that handles remainders runs every time.

```
1  read N
2  if (N <= 0) return ERROR
3  chunks = N / 64;  rem = N % 64
4  for (w = 0; w < chunks; w++)  processBlock(w)    // parallel
5  if (rem > 0)  processPairs(rem / 2)               // BUG: odd rem leaves 1 voxel
6  merge(); return result
```

2. Evaluation of the Reliance on Random Testing

- Random or "round" sizes (256, 512, 1024) have **remainder 0**, so the faulty branch at line 5 is never executed with a meaningful value.
- Odd remainders are only half of all sizes and primes are rare, so random testing hits the failure **by chance** and cannot guarantee it.
- A failing random case is hard to reproduce and gives no coverage measure, so there is no evidence of "enough testing".
- Mathematical code depends on number-theoretic properties (divisibility, parity), which random sampling does not target.

3. Control Flow Graph and Cyclomatic Complexity

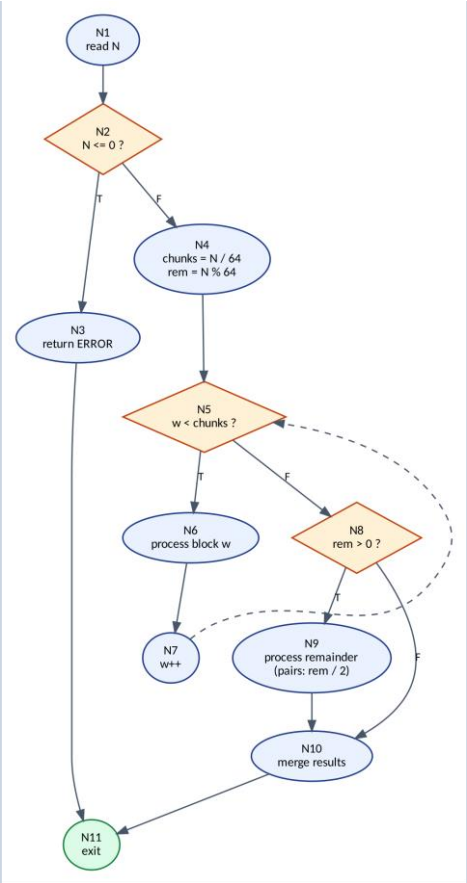


Figure 1: CFG of the parallel-processing module (dashed edge = loop-back)

Item	Value
Nodes (N) / Edges (E)	11 / 13
V(G) = E - N + 2	13 - 11 + 2 = **4**
Predicates (N2, N5, N8) + 1	3 + 1 = **4**

4. Structured Basis Path Testing

Four basis paths are chosen so that each adds a new edge. (The path "loop skipped and rem = 0" is infeasible for N >= 1, which is why it is not listed.)

Path	Node sequence	Test input N	Expected result
P1	N1-N2-N3-N11	0 (and -5)	ERROR returned
P2	N1-N2-N4-N5-N8-N9-N10-N11	61 (prime < 64)	All 61 voxels processed; checksum equals reference
P3	N1-N2-N4-N5-N6-N7-N5-N8-N10-N11	64 (one block, rem 0)	64 voxels processed
P4	N1-N2-N4-N5-N6-N7-N5-N8-N9-N10-N11	97 (prime; 1 block + rem 33)	97 voxels processed; **fails now** (96 done)

5. Additional Targeted Data (mathematical accuracy)

N	chunks / rem	Reason
1	0 / 1	Smallest valid size, odd remainder
2	0 / 2	Prime that is even; remainder even
63, 64, 65	0 / 63, 1 / 0, 1 / 1	Block-size boundary
128, 130	2 / 0, 2 / 2	Even remainders (should pass)
131	2 / 3	Prime, odd remainder
1,000,003	large prime	Stability and memory at scale

Each test compares the processed-voxel count and the **checksum** with a sequential reference implementation (an oracle), and each runs repeatedly with 1, 4 and 16 threads to detect race conditions.

> **Key insight**
> Basis path testing removes the need to guess "special" numbers: P2 and P4 force the remainder branch, and prime values are then used as meaningful data for it.

6. How This Ensures Accuracy and Stability

- Every decision outcome (V(G) = 4 paths) is executed at least once, so the faulty remainder branch can no longer hide.
- The oracle comparison verifies **mathematical accuracy** (no voxel lost or counted twice).
- Repeating the paths with different thread counts checks **stability** (no race condition, no deadlock).
- The path set is small, deterministic and reproducible, so it fits a regression pipeline.

7. Conclusion

Structured **basis path testing** with prime, odd and boundary sizes replaces luck with coverage, locates the odd-remainder defect and gives justified confidence in the module.